POS TECH

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 07 -

DEPLOY E MONITORAMENTO DE CÓDIGO

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

SUMÁRIO

O QUE VEM POR AÍ?.....3
HANDS ON 4
SAIBA MAIS.....7
MERCADO, CASES E TENDÊNCIAS 13
O QUE VOCÊ VIU NESTA AULA? 14
REFERÊNCIAS.....15

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

O QUE VEM POR AÍ?

Quando um protótipo de modelo sai do notebook para produção, a transição do código funcional para um serviço confiável é guiada pela automação: CI valida e empacota, CD promove versões com segurança. Pipelines reprodutíveis (testes, lint, build, rastreabilidade) são essenciais em data science. Os diferentes modos de deploy — batch para grandes volumes e online via API para baixa latência — atendem necessidades distintas do negócio. Estratégias como blue-green e canary reduzem riscos e permitem rollback imediato, enquanto versionar modelos e infraestrutura garante aprendizagem contínua.

O objetivo é construir uma esteira mínima viável (GitHub Actions, Docker, FastAPI) que gera imagem, publica em registro e realiza release progressivo (ex.: Azure). Serão abordadas decisões de arquitetura (container x serverless, gateway, métricas SRE) para a evolução segura dos modelos. Nas videoaulas, você acompanha o fluxo completo e pode pausar para reproduzir os passos.

Colocar em produção é metade do caminho; a outra é garantir performance e confiabilidade. Modelos degradam por drift, ambientes variam e sistemas sob carga revelam problemas nos percentis críticos. A aula conecta observabilidade (traces, métricas, logs) com métricas de ML e rotas de feedback para re-treino e hotfix.

Serão utilizados OpenTelemetry para instrumentação da API, Prometheus para métricas, Application Insights/Azure Monitor como backend gerenciado e Evidently AI para detectar drift. Após o lab das videoaulas, ajuste os limiares e observe os impactos na detecção.

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

HANDS ON

Neste hands on, você verá um modelo scikit-learn empacotado em uma API FastAPI com Docker, testado via Pytest e publicado por GitHub Actions. Abrimos com um panorama rápido do roteiro do vídeo, depois mostramos o código da API, o arquivo Dockerfile e o workflow ci-cd.yml que autentica na nuvem e faz o deploy automatizado.

A gravação enfatiza por que Python é a linguagem escolhida no curso: ampla adoção, excelente ecossistema de ciência de dados e integração nativa com as ferramentas de MLOps usadas. Assista aos vídeos, clone o repositório de exemplo que indicamos e experimente personalizar as etapas para seu contexto.

```python
# app/main.py
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np

app = FastAPI(title="Modelo Demo - FastAPI")

class Features(BaseModel):
    x1: float; x2: float; x3: float

model = joblib.load("model.pkl")

@app.post("/predict")
def predict(f: Features):
    X = np.array([[f.x1, f.x2, f.x3]])
    y = model.predict(X)[0]
    return {"prediction": float(y)}
```

Código-fonte 1 – API FastAPI para servir modelo (Linguagem: Python)
Fonte: Elaborado pelo autor (2025)

```yaml
# .github/workflows/ci-cd.yml
name: ci-cd
on: [push]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.11'}
      - run: pip install -r requirements.txt && pytest -q
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with:
          registry: ${{ secrets.ACR_LOGIN_SERVER }}
          username: ${{ secrets.ACR_USERNAME }}
          password: ${{ secrets.ACR_PASSWORD }}
      - uses: docker/build-push-action@v6
        with:
          context: .
```

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

```yaml
          push: true
          tags: ${{ secrets.ACR_LOGIN_SERVER }}/fastapi-ml:$(echo $GITHUB_SHA | cut -c1-7)
  deploy:
    needs: build-test
    runs-on: ubuntu-latest
    steps:
      - uses: azure/login@v2
        with:
          client-id: ${{ secrets.AZURE_CLIENT_ID }}
          tenant-id: ${{ secrets.AZURE_TENANT_ID }}
          subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
      - uses: azure/cli@v2
        with:
          inlineScript: |
            az containerapp update \
              --name ml-api --resource-group rg-ml \
              --image ${{ secrets.ACR_LOGIN_SERVER }}/fastapi-ml:$(echo $GITHUB_SHA | cut -c1-7)
```

Código-fonte 2 – Workflow de CI/CD com GitHub Actions (Linguagem: YAML)
Fonte: Elaborado pelo autor (2025)

```dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
COPY model.pkl .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Código-fonte 3 – Dockerfile (Linguagem: Dockerfile)
Fonte: Elaborado pelo autor (2025)

Também será realizada a instrumentação da API FastAPI com OpenTelemetry para coletar traces (latência p95/p99) e logs estruturados; em seguida adicionamos Prometheus no endpoint /metrics e finalizamos com um job Python Evidently que compara distribuição de produção vs. referência em janelas horárias, publicando um relatório HTML e um JSON com o status do drift.

Para rodar, configure a variável de conexão/App Insights ou exporte via OTLP para seu coletor — todos os passos aparecem em tela e os links oficiais estão no final do material. Os códigos a seguir exemplificam o que será mostrado em vídeo.

```python
# obs/otel_setup.py
from opentelemetry import trace
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor, ConsoleSpanExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor

def setup_otel(app, service_name="fastapi-ml"):
    provider = TracerProvider(resource=Resource.create({"service.name": service_name}))
    provider.add_span_processor(BatchSpanProcessor(ConsoleSpanExporter()))
    trace.set_tracer_provider(provider)
    FastAPIInstrumentor.instrument_app(app)
```

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

Código-fonte 4 – Instrumentação OTel em FastAPI (Linguagem: Python)
Fonte: Elaborado pelo autor (2025)

```python
# obs/metrics.py
from fastapi import FastAPI, Request
from prometheus_client import Counter, Histogram, make_asgi_app

REQS = Counter("http_requests_total","Total de requisições",["method","path","code"])
LAT = Histogram("http_request_latency_seconds","Latência por rota",["path"])

def setup_metrics(app: FastAPI):
    @app.middleware("http")
    async def _mw(request: Request, call_next):
        path = request.url.path
        with LAT.labels(path).time():
            resp = await call_next(request)
        REQS.labels(request.method, path, resp.status_code).inc()
        return resp
    app.mount("/metrics", make_asgi_app())
```

Código-fonte 5 – Expondo métricas Prometheus (Linguagem: Python)
Fonte: Elaborado pelo autor (2025)

```python
# drift/check_drift.py
import pandas as pd
from evidently.report import Report
from evidently.metrics import DataDriftPreset

ref = pd.read_parquet("data/ref.parquet")
cur = pd.read_parquet("data/current.parquet")

r = Report(metrics=[DataDriftPreset()])
r.run(reference_data=ref, current_data=cur)
r.save_html("reports/drift.html")
open("reports/drift.json","w").write(r.as_dict().__repr__())
```

Código-fonte 6 – Detecção de drift com Evidently (Linguagem: Python)
Fonte: Elaborado pelo autor (2025)

Portanto, não deixe de assistir às videoaulas para compreender o ciclo fim a fim da observabilidade.

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

SAIBA MAIS

Pipelines de CI/CD para ML: o CI/CD em ML amplia o ciclo tradicional com testes estatísticos, controle de dependências e artifacts (dados, modelos e imagens). No CI, além de lint, testes de unidade e segurança, integramos checks de acurácia mínima e comparação com baseline (ex.: ΔAUC ≤ 1 p.p. ou ΔF1 ≤ 0,02), promovendo apenas runs que cumprem o contrato. No CD, tratamos infra como código e versionamos deployments para staging e produção com gates automáticos. No ecossistema Azure, a documentação oficial mostra workflows prontos de GitHub Actions para treinar e publicar modelos no Azure Machine Learning e montar pipelines MLOps v2, incluindo autenticação OIDC (Entra) e promoção por ambientes.

Modos de Deploy (batch vs. online): o Batch maximiza throughput e custo sob latência relaxada (ETL + lote preditivo), enquanto online prioriza p95/p99 baixo com autoscaling horizontal e warm-up de containers. Frameworks leves de serving (FastAPI) e servidores especializados (TensorFlow Serving, TorchServe, BentoML/KServe/Seldon) atendem necessidades distintas: do “faça-você-mesmo” simples ao multiversionamento com autoscaling, mTLS e experiments por peso de tráfego. Comparativos recentes destacam prós/contras de BentoML, KServe e Seldon Core em cenários Kubernetes nativos e foco Python-first.

Estratégias de Lançamento (risco controlado): o Blue-green mantém dois ambientes idênticos e comuta o routing quando o “green” passa nos testes de fumaça; rollback é instantâneo. O Canary envia uma fração progressiva do tráfego para a nova versão (por peso/headers), monitorando latência, error rate e saturação antes de chegar a 100%. Guias com Istio ilustram traffic shifting e políticas de retries/timeouts.

Arquitetura de Referência (Mermaid)

Figura 1 – Pipeline de CI/CD com release progressivo
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

Boas práticas e SLOs: em produção, adotamos SRE Golden Signals (latência, tráfego, erros, saturação) como SLIs, definindo SLOs (ex.: p99 < 300 ms; error rate < 0,5%). Esses sinais guiam canaries e go/no-go no CD.

ASPECTO	BATCH	ONLINE (API)	BLUE-GREEN	CANARY
Objetivo	Throughput	Baixa latência	Zero-downtime com comutação	Rollout gradual com métricas
Risco	Baixo	Médio	Baixo a médio	Baixo (exposição parcial)
Complexidade	Baixa	Média	Média	Alta (roteamento/observabilidade)

Tabela 1 – Modos e estratégias de deploy (resumo)
Fonte: Elaborado pelo autor (2025)

O que diz a tabela?

Tabela 1 – Modos e estratégias de deploy: esta tabela sintetiza quatro abordagens fundamentais para disponibilização de modelos e aplicações em produção: batch, online (API), blue-green e canary. Cada linha representa um eixo crítico de decisão:

- **Objetivo**: o modo batch prioriza throughput, ideal para cenários em que previsões podem ser processadas em lotes (ex.: scoring diário de milhões de registros). Já o online via API atende requisitos de baixa latência, comum em sistemas interativos ou recomendações em tempo real. As estratégias blue-green e canary não alteram o modo de execução, mas definem como novas versões são introduzidas: a primeira troca ambientes inteiros, enquanto a segunda faz rollout gradual.
- **Risco**: o risco operacional é função da exposição. Batch tende a ser menos crítico, pois erros podem ser revertidos antes da próxima execução. Em APIs, qualquer falha impacta usuários imediatamente. Blue-green

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

mitiga risco com rollback instantâneo, enquanto canary reduz impacto ao limitar tráfego inicial, permitindo validação sob carga real.

- **Complexidade**: a simplicidade do batch contrasta com a sofisticação do canary, que exige roteamento dinâmico, métricas e automação para progressão segura. Blue-green demanda duplicação de infraestrutura, mas simplifica rollback. Em pipelines CI/CD, essa complexidade influencia custo e tempo de implementação.

Conclusão técnica: A escolha depende do trade-off entre latência, custo e tolerância a falhas. Em projetos de ciência de dados, recomenda-se iniciar com batch para MVPs e evoluir para APIs com canary quando a criticidade do negócio justificar.

Observabilidade para ML ≠ monitoramento apenas de infraestrutura

A tríade traces-métricas-logs descreve comportamento do serviço, enquanto métricas de modelo refletem a qualidade do aprendizado sob distribuição de produção. Use Golden Signals como SLIs operacionais (latência p95/p99, error rate, RPS, saturação) com SLOs explícitos e incorpore métricas de ML: acurácia real (online/offline), calibration error, drift (PSI, KL, KS), reject rate e cost-aware loss. Esse acoplamento reduz MTTD/MTTR e dá base objetiva para canary e rollbacks.

OpenTelemetry em APIs Python: o OTel padroniza a coleta de traces/metrics/logs de forma vendor-agnostic. Em FastAPI, a instrumentation oficial captura spans HTTP automaticamente, com hooks para atributos de domínio e exportadores OTLP para coletores/serviços (como Azure Monitor). Guias práticos mostram configuração, filtros e zero-code instrumentation quando apropriado.

Prometheus e percentis "verdadeiros": para latência, evite médias e use histogramas com buckets calibrados à sua SLO (ex.: [0.01,0.05,0.1,0.3,0.5,1,2,5]). O scrape do /metrics e recording rules alimentam dashboards (Grafana) e alertas (ex.: latency:p99 > 300ms por 5m). Bibliotecas como prometheus-fastapi-instrumentator aceleram a adoção.

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

Drift: definições e testes

- **Data drift**: mudança da $P(X)$. Detecte com PSI $\sum(A_i - E_i)\ln(A_i/E_i)$, KL $D_{KL}(P \parallel Q) = \sum P(x)\log\frac{P(x)}{Q(x)}$e KS (máxima diferença entre CDFs).
- **Concept drift**: mudança da $P(Y \mid X)$. Requer ground truth atrasado ou proxies (ex.: prediction drift, Δde feature importance). A documentação do Evidently sintetiza quando usar relatórios batch vs. near-real-time e como integrar com pipelines de orquestração.

Integração com Azure Monitor / App InsightsA a distro do Azure para OTel habilita coleta automática e exportação de traces/metrics/logs, além de Live Metrics. Exemplos públicos mostram FastAPI emitindo telemetria para o serviço, com possibilidade de autenticação gerenciada no ambiente.

GOLDEN SIGNAL	KPI DE ML ASSOCIADO	EXEMPLO DE ALERTA
Latência	p95/p99 de /predict	p99 > 300 ms por 5m
Erros	5xx / timeout	error rate > 1% por 10m
Tráfego	RPS / QPS	pico > P95 histórico
Saturação	CPU/GPU, memória	GPU mem > 90% por 10m
Qualidade (extra)	AUC/F1 on-line, PSI	PSI > 0,2 (drift), ΔF1 > 0,03

Tabela 2 – Mapeando Golden Signals para KPIs de ML
Fonte: Elaborado pelo autor (2025)

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

Tabela 2 – Mapeamento dos Golden Signals para KPIs de ML: esta tabela conecta os quatro sinais clássicos de SRE (latência, erros, tráfego, saturação) a métricas específicas de aprendizado de máquina, criando uma visão holística de saúde do sistema:

- **Latência**: percentis p95/p99 do endpoint /predict são cruciais para experiência do usuário. Em ML, latência elevada pode indicar sobrecarga de CPU/GPU ou pré-processamento ineficiente. Alertas devem considerar janelas temporais (ex.: p99 > 300 ms por 5 minutos).
- **Erros**: taxas de 5xx ou timeouts refletem falhas de infraestrutura ou inconsistências no pipeline. Associar isso a métricas de qualidade (ex.: aumento súbito de reject rate) ajuda a diferenciar problemas sistêmicos de degradação do modelo.
- **Tráfego**: RPS/QPS monitoram demanda e suportam decisões de autoscaling. Em ML, picos podem coincidir com eventos sazonais, exigindo ajuste dinâmico de recursos.
- **Saturação**: uso de CPU/GPU e memória indica proximidade do limite físico. Para modelos grandes, saturação de GPU pode gerar latência exponencial. Alertas proativos evitam quedas abruptas.
- **Qualidade (extra)**: métricas como AUC, F1 e PSI (Population Stability Index) complementam sinais operacionais, detectando data drift ou perda de acurácia. Fórmulas como $\mathrm{PSI} = \sum(\mathrm{A_i} - \mathrm{E_i})\ln(\mathrm{A_i}/\mathrm{E_i})$permitem quantificação objetiva do desvio.

Conclusão técnica: integrar Golden Signals com KPIs de ML é essencial para MLOps robusto. Essa correlação viabiliza SLOs que consideram não apenas disponibilidade, mas também performance preditiva, garantindo confiabilidade end-to-end.

Ao longo desta seção, você percebeu que práticas como CI/CD, estratégias de deploy e monitoramento não são apenas ferramentas, mas pilares para garantir confiabilidade e escalabilidade em projetos de ciência de dados. A integração entre automação, observabilidade e métricas específicas de modelos cria um ciclo virtuoso: cada entrega é validada, cada comportamento é medido e cada desvio é

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

corrigido com rapidez. Em essência, engenharia de software aplicada à ciência de dados não é sobre escrever mais código, mas sobre escrever código que evolui com segurança e transparência.

Agora, retorne às videoaulas e conecte teoria e prática: revise os conceitos, execute os exemplos e reflita sobre como essas abordagens podem transformar seus projetos em soluções robustas e sustentáveis. Esse é o caminho para sair do protótipo e alcançar produção com confiança.

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

MERCADO, CASES E TENDÊNCIAS

Tendências em MLOps e CI/CD para 2025

Este artigo apresenta as melhores práticas atuais para pipelines de CI/CD aplicados a Machine Learning, incluindo estratégias de deploy (blue-green, canary), monitoramento com Prometheus/Grafana e detecção de drift com Evidently AI. Destaca ganhos concretos: redução de downtime em até 90% e aceleração de deploys em 60–80%.

Case prático: CI/CD com Azure Databricks

Este conteúdo oficial detalha como integrar pipelines de CI/CD em projetos de ML usando Azure Databricks, abordando DataOps, ModelOps e DevOps. Ele explica como automatizar testes, monitorar qualidade de dados e garantir reprodutibilidade com infraestrutura como código.

Estratégias de Deploy: Blue-Green e Canary Releases

O artigo compara duas abordagens críticas para atualização segura de modelos em produção, explicando benefícios como rollback imediato (blue-green) e rollout gradual com métricas (canary). Inclui passos práticos para implementação e monitoramento.

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Você entendeu como estruturar CI/CD específico para ML, diferenciou batch vs. online, aplicou blue-green/canary com critérios de go/no-go baseados em SLOs e montou uma esteira replicável (build, teste, image push, deploy e promoção). O checkpoint final é praticar o rollback e comprovar que sua observabilidade captura regressões antes de atingir 100% do tráfego.

Você aprendeu a instrumentar traces/métricas/logs com OTel, expor métricas Prometheus, construir relatórios de drift com Evidently e amarrar tudo em SLOs práticos (latência, erro, saturação). O checkpoint final é configurar alertas (p99, error rate, PSI) e validar que um canary apenas é promovido quando todos os SLIs estão verdes.

Ao longo desta aula, você desenvolveu habilidades essenciais para operar esteiras modernas de CI/CD voltadas para projetos de Machine Learning, compreendendo as particularidades entre diferentes tipos de deploy, integrando observabilidade avançada e aplicando boas práticas para garantir qualidade e confiabilidade dos modelos em produção. A adoção de estratégias como blue-green/canary, aliadas ao monitoramento contínuo por meio de métricas e relatórios de drift, elevam o nível de maturidade dos times e reduzem riscos operacionais.

Com o conhecimento adquirido, você está apto(a) a implementar soluções replicáveis e resilientes, promovendo modelos apenas quando há garantia de desempenho, segurança e estabilidade — pilares fundamentais para o sucesso em ambientes de alta cadência e exigência tecnológica.

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

REFERÊNCIAS

CODEZUP. **Implementing Canaries and Blue-Green Deployments in Kubernetes**. 2024. Disponível em: https://codezup.com/implementing-canaries-and-blue-green-deployments-in-kubernetes/. Acesso em: 16 dez. 2025.

EVIDENTLY AI. **Model monitoring for ML in production: a comprehensive guide**. 2025. Disponível em: https://www.evidentlyai.com/ml-in-production/model-monitoring. Acesso em: 16 dez. 2025.

EVIDENTLY AI. **Overview**. 2025. Disponível em: https://docs.evidentlyai.com/docs/platform/monitoring_overview. Acesso em: 16 dez. 2025.

EWASCHUK, R. **Monitoring Distributed Systems**. 2025. Disponível em: https://sre.google/sre-book/monitoring-distributed-systems/. Acesso em: 16 dez. 2025.

GITHUB. **Azure-Samples /opencensus-with-fastapi-and-azure-monitor**. 2025. Disponível em: https://github.com/Azure-Samples/opencensus-with-fastapi-and-azure-monitor. Acesso em: 16 dez. 2025.

GITHUB. **FastAPI + Gunicorn**. 2024. Disponível em: http://prometheus.github.io/client_python/exporting/http/fastapi-gunicorn/. Acesso em: 16 dez. 2025.

GITHUB. **trallnag /prometheus-fastapi-instrumentator**. 2025. Disponível em: https://github.com/trallnag/prometheus-fastapi-instrumentator. Acesso em: 16 dez. 2025.

LEARN. **MLOps (operações de machine learning) de ponta a ponta com o Azure Machine Learning**. 2025. Disponível em: https://learn.microsoft.com/pt-br/training/paths/build-first-machine-operations-workflow/. Acesso em: 16 dez. 2025.

MICROSOFT. **Use GitHub Actions with Azure Machine Learning**. 2025. Disponível em: https://learn.microsoft.com/.../how-to-github-actions-machine-learning. Acesso em: 16 dez. 2025.

OPEN TELEMETRY. **Getting Started**. 2025. Disponível em: https://opentelemetry.io/docs/languages/python/getting-started/. Acesso em: 16 dez. 2025.

PRIYA, B. **Step-by-Step Guide to Deploying Machine Learning Models with FastAPI and Docker**. 2025. Disponível em: https://machinelearningmastery.com/step-by-step-guide-to-deploying-machine-learning-models-with-fastapi-and-docker/. Acesso em: 16 dez. 2025.

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

SOURCE FORGE. **BentoML vs. KServe vs. Seldon Comparison Chart**. 2025. Disponível em: https://sourceforge.net/software/compare/BentoML-vs-KServe-vs-Seldon/. Acesso em: 16 dez. 2025.

VITHLANI, N. **Implementing Blue-Green Deployments with Kubernetes and Istio**. 2025. Disponível em: https://talent500.com/blog/blue-green-deployments-kubernetes-istio/. Acesso em: 16 dez. 2025.

FIAP

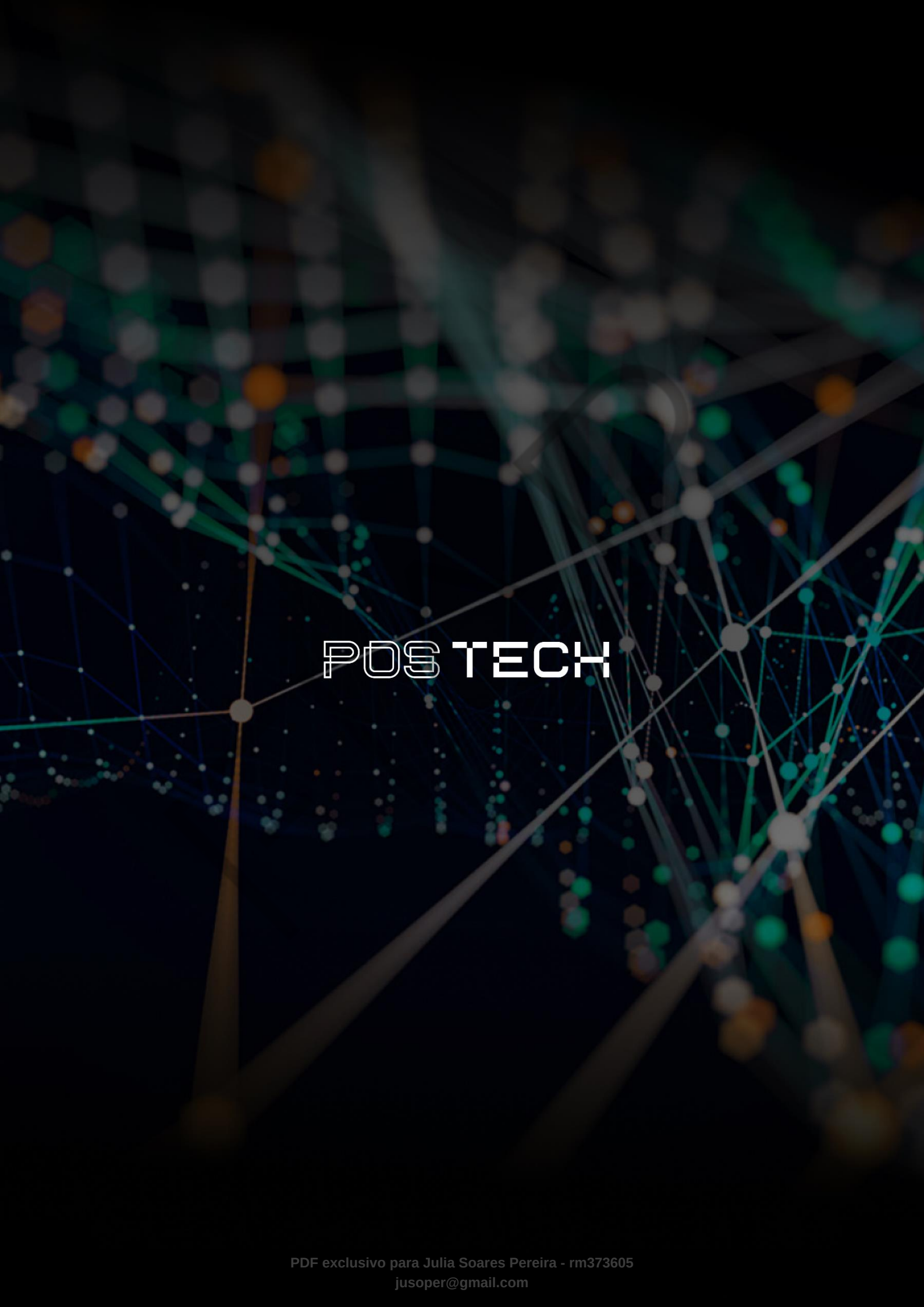
PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com

PALAVRAS-CHAVE

MLOps. Continuous Delivery. SRE.

FIAP

PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com



PDF exclusivo para Julia Soares Pereira - rm373605
jusoper@gmail.com